

Module M214

Application Cloud Native

D.2 — Docker Compose

Gestion d'applications multi-conteneurs

Introduction

Concepts & fichier YAML

Services

App + BDD + RabbitMQ

Gestion

Commandes & cycle de vie

Contents

1	Introduction à Docker Compose	2
1.1	Qu'est-ce que Docker Compose ?	2
1.2	Pourquoi Docker Compose ?	2
1.3	Structure du fichier docker-compose.yml	3
1.4	Les directives clés	4
2	Les Volumes	5
2.1	Pourquoi les volumes sont indispensables ?	5
2.2	Deux types de volumes	5
3	Les Réseaux Docker	6
3.1	Communication entre services	6
4	Exemple Complet : API + MongoDB + RabbitMQ	7
4.1	Architecture de l'exemple	7
4.2	Structure du projet	7
4.3	Le fichier .env	7
4.4	Le Dockerfile de l'API	8
4.5	Le code de l'API	8
4.6	Le fichier docker-compose.yml complet	10
5	Variables d'Environnement et depends_on	12
5.1	Le fichier .env et son utilisation	12
5.2	La directive depends_on	12
6	Commandes Docker Compose	13
6.1	Démarrage et arrêt	13
6.2	Supervision et debug	13
6.3	Reconstruction	14
6.4	Récapitulatif des commandes	15
7	Synthèse	15

1 Introduction à Docker Compose

1.1 Qu'est-ce que Docker Compose ?

Docker Compose

Docker Compose est un outil qui permet de définir et de gérer des **applications multi-conteneurs** à l'aide d'un seul fichier YAML : `docker-compose.yml`.

Au lieu de lancer chaque conteneur séparément avec plusieurs commandes `docker run`, Docker Compose permet de tout démarrer, arrêter et orchestrer en **une seule commande**.

1.2 Pourquoi Docker Compose ?

Dans une application réelle, plusieurs services tournent ensemble : une API, une base de données, un broker de messages... Sans Compose, il faudrait lancer et configurer chaque conteneur manuellement, gérer les réseaux et les dépendances à la main.

Analogie

Si un **Dockerfile** est la recette d'un seul plat, **Docker Compose** est le **menu complet d'un restaurant** : il coordonne tous les plats pour qu'ils soient prêts au bon moment, dans le bon ordre.

Sans Compose

```
docker network create app
```

```
docker run -d -name mongo ...
```

```
docker run -d -name rabbit ...
```

```
docker run -d -name api ...
```

4 commandes, ordre manuel, erreurs fréquentes

Avec Compose

```
docker compose up -d --build
```

1 seule commande, ordre automatique



- ✓ MongoDB démarré en premier
- ✓ RabbitMQ démarré ensuite
- ✓ API démarrée en dernier

1.3 Structure du fichier docker-compose.yml

```
1 version: '3.8'           # Version de la syntaxe Compose
2
3 services:                 # Chaque service = un conteneur
4
5   nom-du-service:
6     image: node:20        # Image Docker Hub
7     # OU
8     build: ./dossier      # Construire depuis un Dockerfile local
9
10    container_name: mon-ctn
11    restart: always       # Redemarrer si crash
12
13    ports:
14      - "3000:3000"       # "port-machine:port-conteneur"
15
16    environment:          # Variables d'environnement
17      - NODE_ENV=production
18      - PORT=3000
19    # OU depuis un fichier .env :
20    env_file:
21      - .env
22
23    depends_on:           # Attendre un autre service
24      - mongodb
25
26    volumes:              # Monter un dossier ou un volume
27      - ./data:/app/data  # Dossier local
28      - mongo-data:/data/db # Volume nomme
29
30    networks:
31      - app-network
32
33 volumes:                 # Volumes persistants
34   mongo-data:
35
36 networks:                # Reseau interne
37   app-network:
38     driver: bridge
```

Listing 1: Structure générale d'un docker-compose.yml

1.4 Les directives clés

Directive	Rôle
<code>image</code>	Image Docker à utiliser (depuis Docker Hub)
<code>build</code>	Chemin vers le Dockerfile pour construire l'image
<code>container_name</code>	Nom du conteneur (pratique pour les logs et exec)
<code>ports</code>	Redirection de ports <code>machine:conteneur</code>
<code>environment</code>	Variables d'environnement du conteneur
<code>env_file</code>	Charger les variables depuis un fichier <code>.env</code>
<code>depends_on</code>	Ordre de démarrage (ce service démarre après...)
<code>volumes</code>	Persistance des données ou partage de fichiers
<code>networks</code>	Réseau interne pour la communication entre services
<code>restart</code>	Politique de redémarrage (<code>always</code> , <code>on-failure</code> ...)

2 Les Volumes

2.1 Pourquoi les volumes sont indispensables ?

✖ Sans volume : les données sont perdues !

Par défaut, quand un conteneur est supprimé, **toutes ses données disparaissent** avec lui. Si MongoDB tourne dans un conteneur sans volume, toutes les données de la base sont effacées à chaque `docker compose down`.

Volume Docker

Un **volume** est un espace de stockage géré par Docker, **indépendant du cycle de vie du conteneur**. Les données persistent même si le conteneur est supprimé et recréé.

2.2 Deux types de volumes

Volume Nommé

`mongo-data:/data/db`

Géré par Docker
Déclaré dans `volumes:`
Portable

Bind Mount

`./config:/app/config`

Dossier local monté
Modifiable en temps réel
Développement

```

1 services:
2   mongodb:
3     image: mongo:7.0
4     volumes:
5       # Volume nommé : Docker gère l'emplacement
6       - mongo-data:/data/db
7
8   api:
9     build: ./api
10    volumes:
11      # Bind mount : le dossier local ./api est monté dans /app
12      # Utile en développement pour voir les changements sans
13      rebuild
14      - ./api:/app
15
16 # Toujours déclarer les volumes nommés ici
17 volumes:
18   mongo-data:
```

Listing 2: Déclaration des deux types de volumes

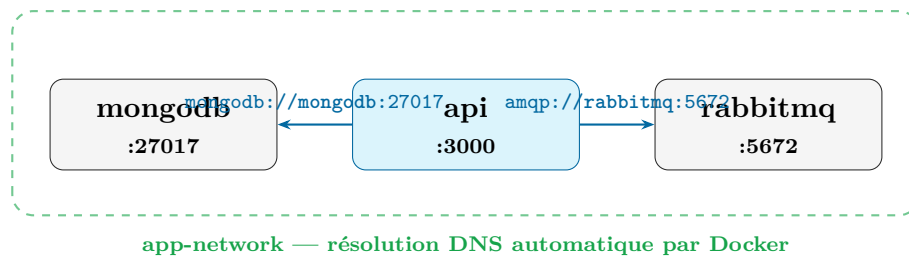
3 Les Réseaux Docker

3.1 Communication entre services

Réseau Docker interne

Dans Docker Compose, tous les services déclarés sur le même réseau peuvent se parler **en utilisant le nom du service comme nom d'hôte**.

Un service n'a **jamais** besoin de connaître l'adresse IP d'un autre service : le réseau Docker fait la résolution de noms automatiquement.



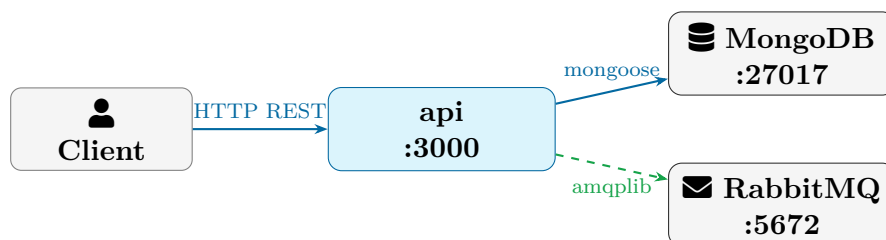
⚠ Ne jamais utiliser localhost entre services !

- `localhost` dans un conteneur pointe vers **lui-même**, pas vers les autres services.
- Pour accéder à MongoDB depuis l'API :
 - `mongodb://mongodb:27017` ✓
 - `mongodb://localhost:27017` ✗
- Pour accéder à RabbitMQ depuis l'API :
 - `amqp://rabbitmq:5672` ✓
 - `amqp://localhost:5672` ✗

4 Exemple Complet : API + MongoDB + RabbitMQ

4.1 Architecture de l'exemple

Nous allons construire une application composée de trois services :



Flèche pleine = sync Flèche pointillée = async

4.2 Structure du projet

```

1 mon-projet/
2   |-- api/
3     |   |-- app.js
4     |   |-- package.json
5     |   |-- Dockerfile
6   |-- .env
7   |-- docker-compose.yml
  
```

Listing 3: Arborescence du projet

4.3 Le fichier .env

```

1 # MongoDB
2 MONGO_ROOT_USER=admin
3 MONGO_ROOT_PASS=admin123
4
5 # RabbitMQ
6 RABBITMQ_USER=admin
7 RABBITMQ_PASS=admin123
  
```

Listing 4: .env — variables sensibles

⚠ Ne jamais commiter le fichier .env !

Ajoutez .env dans votre .gitignore pour ne pas exposer vos mots de passe dans un dépôt Git public.

4.4 Le Dockerfile de l'API

```
1 FROM node:20-alpine
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install --only=production
5 COPY . .
6 EXPOSE 3000
7 CMD ["node", "app.js"]
```

Listing 5: api/Dockerfile

4.5 Le code de l'API

```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const amqp = require('amqplib');
4
5 const app = express();
6 app.use(express.json());
7
8 // Connexion MongoDB (le nom "mongodb" = nom du service Compose)
9 mongoose.connect(process.env.MONGO_URI)
10   .then(() => console.log('MongoDB connecte'))
11   .catch(err => console.error('Erreur MongoDB:', err.message));
12
13 // Schema Mongoose
14 const produitSchema = new mongoose.Schema({
15   nom: { type: String, required: true },
16   prix: { type: Number, required: true }
17 }, { timestamps: true });
18
19 const Produit = mongoose.model('Produit', produitSchema);
20
21 // GET tous les produits
22 app.get('/api/produits', async (req, res) => {
23   const produits = await Produit.find();
24   res.json(produits);
25 });
26
27 // POST creer un produit + publier un evenement RabbitMQ
28 app.post('/api/produits', async (req, res) => {
29   try {
30     const produit = await Produit.create(req.body);
31
32     // Publier un evenement asynchrone dans RabbitMQ
33     const conn = await amqp.connect(process.env.
RABBITMQ_URL);
34     const channel = await conn.createChannel();
35     await channel.assertQueue('produits', { durable: true });
36     channel.sendToQueue(
37       'produits',
```

```
38         Buffer.from(JSON.stringify({ type: 'PRODUIT_CREE',
39         data: produit })),
40         { persistent: true }
41     );
42     await channel.close();
43     await conn.close();
44
45     res.status(201).json(produit);
46 } catch (err) {
47     res.status(400).json({ error: err.message });
48 }
49 });
50 app.listen(process.env.PORT || 3000,
51     () => console.log('API sur port', process.env.PORT || 3000));
```

Listing 6: api/app.js

4.6 Le fichier docker-compose.yml complet

```

1 version: '3.8'
2
3 services:
4
5     # =====
6     # Base de donnees : MongoDB
7     # =====
8     mongodb:
9         image: mongo:7.0
10        container_name: mongodb
11        restart: always
12        environment:
13            MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USER}
14            MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASS}
15        ports:
16            - "27017:27017"
17        volumes:
18            - mongo-data:/data/db          # Donnees persistantes
19        networks:
20            - app-network
21
22    # =====
23    # Message Broker : RabbitMQ
24    # =====
25    rabbitmq:
26        image: rabbitmq:3-management
27        container_name: rabbitmq
28        restart: always
29        environment:
30            RABBITMQ_DEFAULT_USER: ${RABBITMQ_USER}
31            RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASS}
32        ports:
33            - "5672:5672"                  # Port AMQP (connexion
34            - "15672:15672"                # Interface web de gestion
35        volumes:
36            - rabbitmq-data:/var/lib/rabbitmq
37        networks:
38            - app-network
39
40    # =====
41    # API Node.js
42    # =====
43    api:
44        build: ./api                      # Construit depuis api/Dockerfile
45        container_name: api
46        restart: always
47        environment:
48            - PORT=3000
49            - MONGO_URI=mongodb://${MONGO_ROOT_USER}:${MONGO_ROOT_PASS}
50            @mongodb:27017/produitsdb?authSource=admin

```

```
50     - RABBITMQ_URL=amqp://${RABBITMQ_USER}:${RABBITMQ_PASS}
    @rabbitmq:5672
51     depends_on:
52     - mongodb
53     - rabbitmq
54     ports:
55     - "3000:3000"
56     networks:
57     - app-network
58
59     # =====
60     # Volumes persistants
61     # =====
62     volumes:
63     mongo-data:
64     rabbitmq-data:
65
66     # =====
67     # Réseau interne
68     # =====
69     networks:
70     app-network:
71     driver: bridge
```

Listing 7: docker-compose.yml — Architecture complète

5 Variables d'Environnement et depends_on

5.1 Le fichier .env et son utilisation

```
1 # Dans .env :
2 DB_PORT=27017
3
4 # Dans docker-compose.yml :
5 ports:
6   - "${DB_PORT}:27017"
7
8 # Docker Compose lit automatiquement le fichier .env
9 # si il est dans le meme dossier que docker-compose.yml
```

Listing 8: Utilisation des variables .env dans docker-compose.yml

5.2 La directive depends_on

depends_on

`depends_on` définit l'ordre de démarrage des services. Dans l'exemple ci-dessous, `api` ne démarre qu'après `mongodb` et `rabbitmq`.

⚠ Limite de depends_on

`depends_on` garantit que le **conteneur est démarré**, mais **pas** que le service est prêt à accepter des connexions. MongoDB peut mettre quelques secondes à être pleinement opérationnel même après le démarrage du conteneur.

Solution : gérer les erreurs de connexion avec une logique de **rétry** dans le code de l'API (tentative toutes les X secondes).

```
1 async function connectWithRetry(maxRetries = 10) {
2   for (let i = 1; i <= maxRetries; i++) {
3     try {
4       await mongoose.connect(process.env.MONGO_URI);
5       console.log('MongoDB connecte');
6       return;
7     } catch (err) {
8       console.log(`Tentative ${i}/${maxRetries} echouee.
9       Retry dans 3s...`);
10      await new Promise(r => setTimeout(r, 3000));
11    }
12    throw new Error('Connexion MongoDB impossible');
13  }
14
15  connectWithRetry();
```

Listing 9: Gestion de la reconnexion dans Node.js

6 Commandes Docker Compose

6.1 Démarrage et arrêt

```
# Construire les images ET démarrer tous les services
docker compose up --build

# Démarrer en arrière-plan (mode detache)
docker compose up -d --build

# Démarrer uniquement certains services
docker compose up -d mongodb rabbitmq

# Arrêter tous les services (sans supprimer les données)
docker compose stop

# Arrêter ET supprimer conteneurs + réseau (données conservées)
docker compose down

# Arrêter ET supprimer TOUT, y compris les volumes (données
  perdues !)
docker compose down -v
```

Listing 10: Commandes de base

 **down -v supprime toutes les données !**

L'option **-v** supprime les volumes nommés déclarés dans `docker-compose.yml`. Toutes les données MongoDB et RabbitMQ seront effacées. À utiliser uniquement pour un reset complet.

6.2 Supervision et debug

```
# Etat de tous les services
docker compose ps

# Logs en temps réel (tous les services)
docker compose logs -f

# Logs d'un service spécifique
docker compose logs -f api
docker compose logs -f mongodb

# Entrer dans un conteneur (shell interactif)
docker compose exec api sh
docker compose exec mongodb mongosh -u admin -p admin123

# Voir la consommation CPU / mémoire
docker stats
```

Listing 11: Surveiller l'état des services

6.3 Reconstruction

```
# Reconstruire l'image d'un service apres modification du code  
docker compose build api  
  
# Redemarrer un service (sans rebuild)  
docker compose restart api  
  
# Reconstruire ET redemarrer en une commande  
docker compose up -d --build api
```

Listing 12: Rebuild et restart

6.4 Récapitulatif des commandes

Commande	Description
<code>docker compose up -d -build</code>	Construire et démarrer tous les services
<code>docker compose down</code>	Arrêter et supprimer conteneurs + réseau
<code>docker compose down -v</code>	Idem + supprimer les volumes
<code>docker compose stop</code>	Arrêter sans supprimer
<code>docker compose start</code>	Redémarrer des services arrêtés
<code>docker compose ps</code>	État de tous les services
<code>docker compose logs -f</code>	Logs en temps réel
<code>docker compose logs -f svc</code>	Logs d'un service spécifique
<code>docker compose build svc</code>	Reconstruire l'image d'un service
<code>docker compose restart svc</code>	Redémarrer un service
<code>docker compose exec svc sh</code>	Shell interactif dans un conteneur
<code>docker stats</code>	Consommation CPU/mémoire en temps réel

7 Synthèse

Points clés à retenir

1. **docker-compose.yml** décrit toute l'architecture en un fichier YAML
2. Chaque **service** correspond à un conteneur avec sa propre configuration
3. Les services communiquent via leur **nom de service** (jamais `localhost`)
4. Les **volumes nommés** assurent la persistance des données
5. **depends_on** contrôle l'ordre de démarrage
6. Le fichier **.env** stocke les variables sensibles (ne pas commiter !)
7. `docker compose up -d -build` est la commande de référence
8. `docker compose down -v` supprime tout, y compris les données

Outil	Rôle	Fichier clé
docker-compose.yml	Définir l'architecture complète	docker-compose.yml
.env	Variables d'environnement sensibles	.env
Volumes	Persistance des données	Déclarés dans volumes:
Networks	Communication interne	Déclarés dans networks:
depends_on	Ordre de démarrage	Clé du service